Formattazione dei Numeri

```
double x = 1.3546789;
printf("x = %f \n", x)
printf("x = %lf \n", x)
```

```
float y = -1.34;
printf("y = %f \n", y)
```

tipicamente usa / stampa 6 cifre decimali

`%n.m(tipo)`

n: # minimo di caratteri/numeri: incluso eventuale . decimale

m: # num. di cifre decimali

```
printf("x = %.2f \n", x)
```

`%5.2f`

x = 1.36

formati.c

```c
#include <stdio.h>

int main() {
    double x = 12345678.123456789;
    double y = 0.0001234567;
    printf("formato x\t\ty\n");
    printf("%%f      %f\t%f\n", x, y);
    printf("%%5.3f   %5.3f\t%5.3f\n", x, y);
    printf("%%5.3e   %5.3e\t%5.3e\n", x, y);
    printf("%%5.3g   %5.3g\t%5.3g\n", x, y);
    printf("%%.15f   %.15f\t%.15f\n", x, y);
}
```

\n : vai a capo

\t : tab

caratteri a destra per incolonnare output

% carattere speciale per la formattazione

printf("30%%\n");

30%

Terminale: formati.c

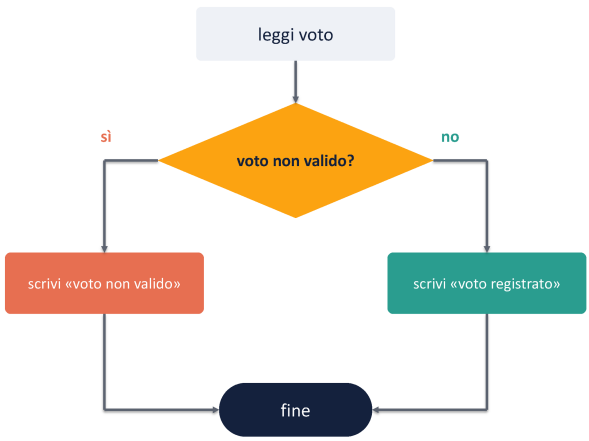
```
$ gcc -Wall -Wextra -o app.exe formati.c -lm
$ ./app.exe
formato x                   y
%f      12345678.123457 0.000123
%5.3f   12345678.123    0.000
%5.3e   1.235e+07       1.235e-04
%5.3g   1.23e+07        0.000123
%.15f   12345678.123456789180636        0.000123456700000
```

Flusso del programma / Decisioni

voto.c

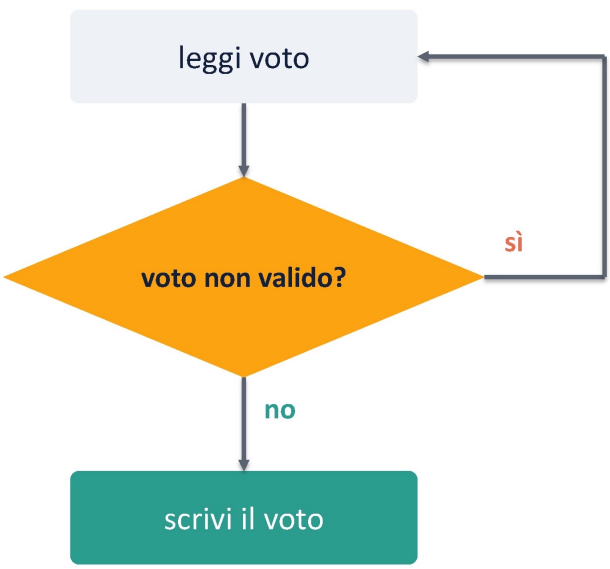
```c
#include <stdio.h>

int main() {
    int voto;
    printf("voto (0-30): ");
    scanf("%d", &voto);
    if (voto < 0 || voto > 30) {
        printf("voto non valido\n");
    } else {
        printf("voto %d registrato\n", voto);
    }
}
```



If/else non si può tornare indietro nel programma

Flusso desiderato



Ciclo do { } while ( );

```
do {
    leggi dati / voti
} while ( condizione per ripetere è vera );
    voto < 0 || voto > 30
```

voto.c

```c
#include <stdio.h>

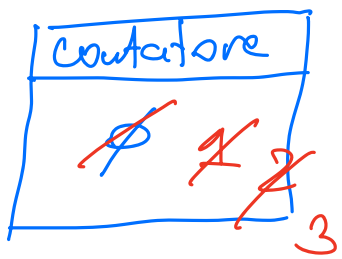
int main() {
    int voto;
    do {
        printf("voto (0-30): ");
        scanf("%d", &voto);
    } while (voto < 0 || voto > 30);
    printf("voto %d registrato\n", voto);
}
```

Terminale: voto.c

```
$ gcc -Wall -Wextra -o app.exe voto.c -lm
$ ./app.exe
voto (0-30): 45
voto (0-30): -3
voto (0-30): 27
voto 27 registrato
```

Contatore = quante iterazioni del ciclo?

```
int voto;
int contatore = 0;   // fuori dal ciclo
do {
    contatore = contatore + 1;
    printf(" voto -- ");
    scanf("%d", &voto)
} while( -- );
printf(" iterazioni : %d \n", contatore );
```

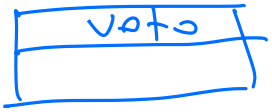


tentativi.c

```c
#include <stdio.h>

int main() {
    int voto = -1;          // -1 = non valido
    int ntentativi = 0;
    do {
        printf("voto (0-30): ");
        scanf("%d", &voto);
        ntentativi = ntentativi + 1;
    } while (voto < 0 || voto > 30);
    printf("voto %d registrato\n", voto);
    printf("tentativi: %d\n", ntentativi);
}
```

buona pratica inizializzare




```
Terminale: tentativi.c

$ gcc -Wall -Wextra -o app.exe tentativi.c -lm
$ ./app.exe
voto (0-30): 45
voto (0-30): -3
voto (0-30): 27
voto 27 registrato
tentativi: 3
$ ./app.exe
voto (0-30): 28
voto 28 registrato
tentativi: 1
```

Errore comune: condizione invertita.

```
do {
    ...
} while (voto > 0 && voto <= 30);
```

non deve uscire

ripete solo per valori validi:
accetta il primo voto non valido!

do-while: fa sempre almeno 1 iterazione

ntentativi >= 1;

ntentativi = ntentativi + 1;

ntentativi++;

double x = 0.5;
x++; x: 0.5 → 1.5

n = n+1 ⟹ n++;

n = n-1; n--;

n = n+5; ⟷ n+=5 equivalenti

int contatore = 0;

do {

≡ ciclo

contatore++;

} while (cond. ripetizione vera);

do-while → sempre almeno 1 tentativo.

Serve con ciclo con — prima controllo
— poi iterazione

Ciclo while

while(Condizione vera) {

≡

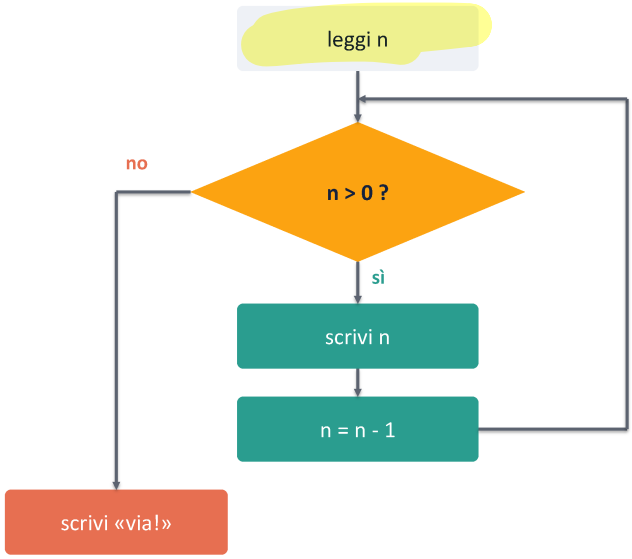
}

Ciclo while

conto.c

```c
#include <stdio.h>

int main() {
    int n;
    printf("n = ");
    scanf("%d", &n);
    while (n > 0) {
        printf("%d\n", n);
        n = n - 1;
    }
    printf("via!\n");
}
```



Terminale: conto.c

```
$ gcc -Wall -Wextra -o app.exe conto.c -lm
$ ./app.exe
n = 3
3
2
1
via!
$ ./app.exe
n = 0
via!
```

do { ... } while (condizione);

Controllo dopo prima iterazione

while (condizione) { ... }

Controllo prima di ogni iterazione

Errore Concettuale

conto.c

```c
#include <stdio.h>

int main() {
    int n;
    printf("n = ");
    scanf("%d", &n);
    while (n > 0);
    {
        printf("%d\n", n);
        n = n - 1;
    }
    printf("via!\n");
}
```

Terminale: conto.c

```
$ gcc -Wall -Wextra -o app.exe conto.c -lm
conto.c:7:18: warning: while loop has empty body [-Wempty-body]
$ ./app.exe
n = 3
^C    (il ciclo non finisce: n non cambia mai)
```

; non ci deve essere

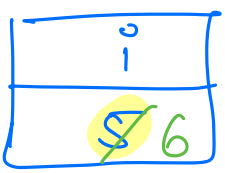
; non serve

Nuovi Operatori

abbreviazioni.c

```c
#include <stdio.h>

int main() {
    int i = 5;
    i++;                    // come i = i + 1
    printf("i++   %d\n", i);
    i += 4;                 // come i = i + 4
    printf("+= 4  %d\n", i);
    i *= 3;                 // come i = i * 3
    printf("*= 3  %d\n", i);
    i--;                    // come i = i - 1
    printf("i--   %d\n", i);
    i -= 9;                 // come i = i - 9
    printf("-= 9  %d\n", i);
}
```



ciao

esiste anche

i /= 3

i = i / 3;

ovvio i++;

i += 1;

binario

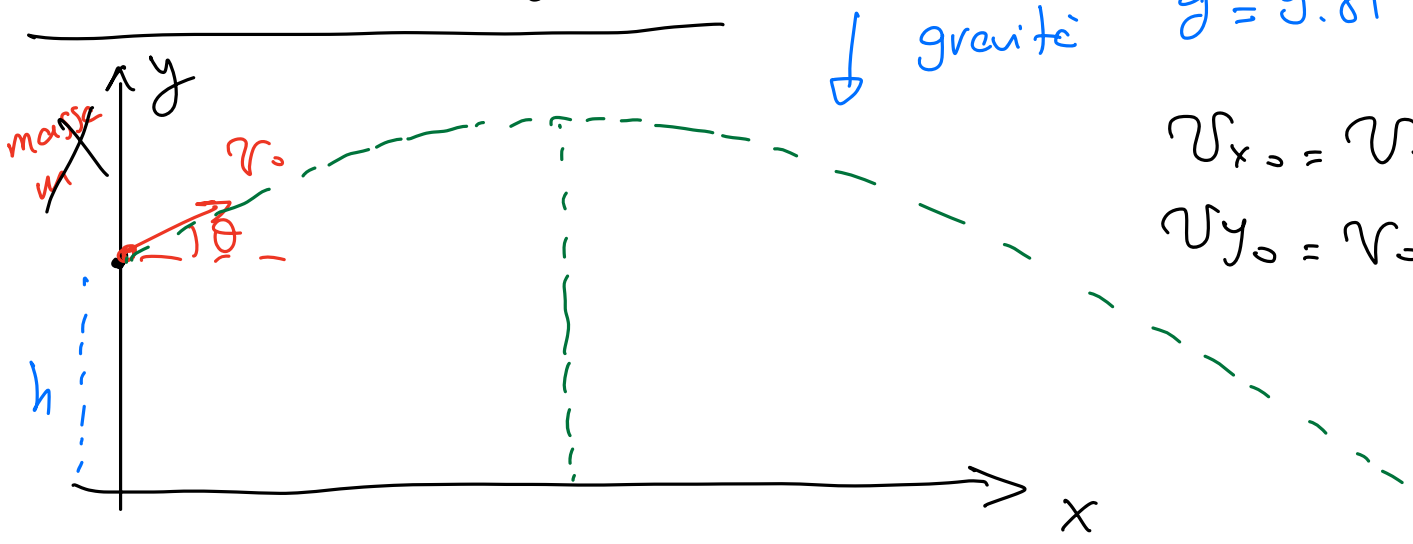
printf(" i += 4 %d \n", i)

Terminale: abbreviazioni.c

```
$ gcc -Wall -Wextra -o app.exe abbreviazioni.c -lm
$ ./app.exe
i++   6
+= 4  10
*= 3  30
i--   29
-= 9  20
```

i += 4 10

Moto di un grave



$g = 9.81\ m/s^2$

$v_{x_0} = v_0 \cos\theta$

$v_{y_0} = v_0 \sin\theta$

$$x(t) = x_0 + v_{x_0} \cdot t$$

$$y(t) = h - \frac{1}{2} g t^2 - v_{y_0} t$$

$v_0 = 0 \Rightarrow$ Caduta del grave

$t = 0 \qquad x = 0 \qquad y = h$

$t = \Delta t \qquad \Delta t = 0.1\ s.$

$t = 2\Delta t \qquad \Delta t = 0.001\ s$

$t = 3\Delta t$

```
double dt = 0.1;     // secondi
double grav = 9.81;  // m/s/s
```

$$y(t) = h - \frac{1}{2} g t^2$$

$$y(t+\Delta t) = h - \frac{1}{2} g (t+\Delta t)^2$$

Dichiarare costanti con il precompilatore

#define GRAV 9.81

(␣ spazio, ␣ spazio)

nessun ;

perché non è sintassi C

Caduta del grave :

while oppure do_while ?

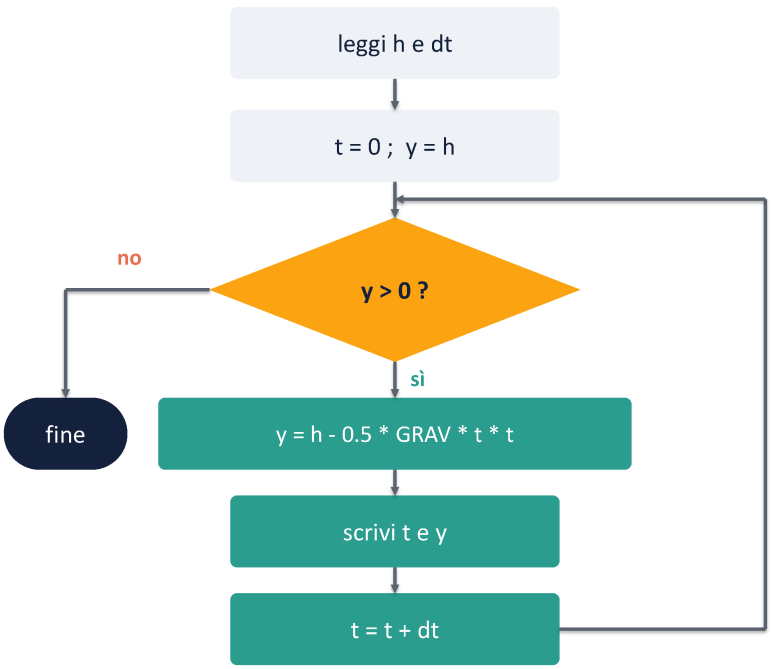
condizione per calcolo del

passo successivo : $y > 0$

⇒ while (y > 0) {

≡

}



grave.c

```c
#include <stdio.h>
#define GRAV 9.81

int main() {
    double h, dt, t, y;
    printf("altezza h (m): ");
    scanf("%lf", &h);
    printf("passo dt (s): ");
    scanf("%lf", &dt);
    t = 0.;
    y = h;
    printf("   t\t   y\n");
    while (y > 0.) {
        y = h - 0.5 * GRAV * t * t;
        printf("%6.2f\t%6.2f\n", t, y);
        t += dt;
    }
}
```

if (y>0) printf(...)

Terminale: grave.c

```
$ gcc -Wall -Wextra -o app.exe grave.c -lm
$ ./app.exe
altezza h (m): 20
passo dt (s): 0.5
   t        y
  0.00    20.00
  0.50    18.77
  1.00    15.09
  1.50     8.96
  2.00     0.38
  2.50   -10.66
```